



Optimización de Rendimiento en Bases de Datos

Dr. Eric Gustavo Coronel Castillo

DOCENTE

RESULTADO DEL APRENDIZAJE

Al finalizar la sesión, el estudiante será capaz de analizar planes de ejecución de consultas, identificar ineficiencias y aplicar técnicas de optimización, incluyendo el impacto de los parámetros físicos de almacenamiento, justificando cada decisión con criterios técnicos.

Saber

- Conceptos de plan de ejecución
- Tipos de acceso a datos
- Parámetros físicos: block, extent, freespace

Saber hacer

- Leer un plan de ejecución
- Crear índices adecuados
- Reescribir consultas SQL ineficientes

Saber decidir

- Evaluar trade-offs (índice vs. overhead)
- Elegir parámetros físicos óptimos
- Aplicar desnormalización controlada

MOTIVACIÓN

MOTIVACIÓN

¿Por qué el “rendimiento” importa tanto?

Experiencia de usuario

Una consulta lenta se siente como “el sistema está caído”. En operaciones, segundos importan.

Costo y escalabilidad

Más CPU/RAM no siempre resuelve si el problema es E/S o un mal plan. Optimizar puede evitar hardware.

Riesgo operativo

Cambios sin medir (índices/hints) pueden mejorar una consulta y empeorar el sistema completo.

Idea central

En la mayoría de SGBD relacionales, el cuello de botella típico es el acceso a datos: buffers ↔ disco. El tuning se enfoca en reducir trabajo (E/S, filas) y elegir mejores rutas de acceso (planes, índices, estadísticas).

SABERES PREVIOS

SABERES PREVIOS

¿Qué es un índice en una base de datos y para qué sirve?

A

Es una copia de seguridad compacta de la tabla que se almacena en la memoria RAM para evitar que la base de datos se bloquee cuando hay muchos usuarios conectados simultáneamente.

B

Es una estructura de datos que almacena punteros a las filas de una tabla, permitiendo localizar registros rápidamente (Index Seek) sin tener que recorrer toda la tabla (Full Table Scan).

C

Es una restricción de integridad que obliga a que los campos de una tabla sean numéricos, optimizando así el espacio en disco y acelerando los cálculos matemáticos en las consultas.

SABERES PREVIOS

¿Qué significa que una consulta haga **full table scan**?

A

Que el motor de búsqueda lee secuencialmente todas las filas de la tabla para encontrar los registros solicitados.

B

Que la base de datos ha alcanzado su capacidad máxima de almacenamiento y necesita expandir el espacio en disco para completar la consulta.

C

Que se está realizando una copia de seguridad completa de la tabla para proteger la integridad de los datos antes de ejecutar una actualización.

DESARROLLO

MAPA DE 4 IDEAS

1

Medir antes de tocar

Sin línea base, “mejoró” es una ilusión.

Define métrica mínima: tiempo, filas, lecturas.

2

Leer el plan de ejecución

El optimizador decide: accesos (scan/seek) + joins + orden de operaciones.

3

Índices + estadísticas

Índices dan rutas; estadísticas guían decisiones. Si fallan, el plan se degrada.

4

Cambiar con cautela

Reescribir SQL, ajustar índices, actualizar estadísticas.

Hints: último recurso.

Idea 1: El **costo** suele estar dominado por E/S (disco) y buffers

Ruta simplificada para obtener datos

- 1) Verificar si el dato ya está en memoria (buffer/cache)
- 2) Si no está: ubicarlo en disco (acceso físico)
- 3) Transferirlo a memoria para procesarlo

Conclusión práctica: reducir E/S y filas leídas suele dar el mayor impacto.

Dónde se pierde tiempo



Idea 2: Leer un **plan de ejecución** (lo mínimo indispensable)

Señales típicas en el plan

- Tipo de acceso: scan (barrido) vs seek/scan por índice
- Cardinalidad: filas estimadas vs filas reales
- Métodos de JOIN: nested loops / hash join / merge join
- Orden de operaciones (árbol del plan)
- Operadores caros: sort, hash, agregaciones

Regla de oro: el plan es un “mapa” del trabajo que hará el motor.

Esquema: plan como árbol

```
Hash Join
├─ Seq/Index Scan (Tabla A)
└─ Index Scan (Tabla B)
```

Lectura: de abajo hacia arriba (los hijos alimentan al padre).

Idea 2: Leer un **plan de ejecución** (lo mínimo indispensable)

Métodos de JOIN

Característica	Nested Loop	Hash Join	Merge Join
¿Necesita orden previo?	No	No	Si, ambas tablas
¿Necesita índice?	Sí, en la tabla interna	No	Ayuda, pero no obliga
Uso de memoria	Bajo	Alto	Bajo
Mejor escenario	Tabla externa pequeña con índice sobre la columna de JOIN en la tabla interna	Tablas grandes sin orden ni índice	Tablas grandes ya ordenadas
Riesgo principal	Muchas iteraciones si la tabla externa es grande	Se va a disco si la memoria no alcanza	Costo extra si hay que ordenar primero
Consulta típica	OLTP · Transaccional	OLAP · Reportes	Ambos · Con índices

Idea 3: Índices y estadísticas

Por qué a veces **está todo indexado**, igual es lento

Índices (rutas)

- Aceleran búsquedas/joins al reducir lecturas
- Pueden ser compuestos (varias columnas)
- Trade-off: cada escritura (INSERT/UPDATE/DELETE) también debe mantener índices
- Índices innecesarios consumen espacio y CPU

Estadísticas (guía del optimizador)

- Resumen de distribución de datos (filas, selectividad, etc.)
- Si están desactualizadas → estimaciones malas → planes malos
- La recomendación general es mantenerlas actualizadas según cambios de datos
- Hints/directivas: usar con precaución (pueden “forzar” planes frágiles).

Idea 4: Cambiar con cautela

Reescritura de consultas

No recomendado ❌	Recomendado ✅
Función en columna indexada invalida el índice <pre>SELECT * FROM pedidos WHERE YEAR(fecha_pedido) = 2024;</pre>	Equivalente optimizado, permite usar índice <pre>SELECT * FROM pedidos HERE fecha_pedido ≥ '2024-01-01' AND fecha_pedido < '2025-01-01';</pre>
Usar SELECT * trae columnas innecesarias <pre>SELECT * FROM empleados WHERE depto_id = 10;</pre>	Solo las columnas necesarias <pre>SELECT codigo, nombre, salario FROM empleados WHERE depto_id = 10;</pre>
Usar NOT IN con subconsulta puede ser lento <pre>SELECT * FROM clientes WHERE id NOT IN (SELECT cliente_id FROM pedidos);</pre>	Equivalente con NOT EXISTS más eficiente <pre>SELECT * FROM clientes c WHERE NOT EXISTS (SELECT 1 FROM pedidos p WHERE p.cliente_id = c.id);</pre>
OR en columna indexada puede forzar full scan <pre>SELECT * FROM empleados WHERE depto_id = 1 OR depto_id = 2;</pre>	IN suele ser mas amigable para el optimizador <pre>SELECT * FROM empleados WHERE depto_id IN (1, 2);</pre>

- NOT EXISTS evita problemas con NULL y suele optimizar mejor.
- Siempre validar con plan de ejecución y métricas antes/después; evitar cambios sin evidencia.

Parámetros Físicos: Block, Extent y Freespace

Principio fundamental: Los parámetros físicos de almacenamiento tienen impacto directo en el rendimiento. No son solo configuración administrativa: son decisiones de diseño de rendimiento. (Reyna Monteverde, pp. 56-98)



Block Size (Tamaño de bloque)

- Unidad mínima de I/O entre disco y RAM
- **OLTP / Online:** bloques pequeños (2K–4K) → más bloques en buffer, mayor concurrencia
- **Batch / DWH:** bloques grandes (16K–32K) → menos I/O, mayor throughput



Extents (Áreas de Control)

- Unidad de asignación de espacio para el crecimiento de un objeto (tabla/índice)
- Usar tamaños de extent adecuados para el crecimiento esperado
- Fragmentación → más I/O aleatorio y menos lectura secuencial → degradación del rendimiento.



Freespace (Espacio libre)

- Porcentaje reservado en cada bloque para crecimiento futuro
- **%PCTFREE (nivel bloque):** espacio para updates que aumentan el tamaño del registro
- **%PCTUSED / %AC (nivel extent):** IC vacíos para nuevas inserciones
- **Sin freespace** → rupturas de bloque → fragmentación → lentitud

EVALUACIÓN

QUE HEMOS APRENDIDO

¿Por qué aplicar una función sobre una columna indexada en el WHERE puede degradar el rendimiento?

- A. Porque el optimizador puede no usar el índice de forma eficiente y terminar leyendo más filas o páginas.
- B. Porque una función en el WHERE siempre provoca un bloqueo exclusivo de la tabla durante la consulta.
- C. Porque el uso de funciones en el WHERE convierte automáticamente la consulta en una transacción distribuida.

QUE HEMOS APRENDIDO

Una tabla tiene 10 millones de registros. Se realizan 500,000 inserciones diarias y 10 consultas por segundo. ¿Crearías muchos índices o pocos?

- A. Muchos índices, entre más índices, siempre mejora rendimiento.
- B. Crear índices para todas las columnas, así el optimizador nunca hará full scan.
- C. Pocos índices, los estrictamente necesarios y alineados a las 10 consultas por segundo.

CONCLUSIONES

CONCLUSIONES

- Rendimiento se mejora reduciendo trabajo (E/S, filas) y eligiendo mejores rutas (índices).
- El plan de ejecución es la “verdad operativa” de cómo se ejecuta la consulta.
- Índices y estadísticas son complementarios: rutas + guía.
- Cambiar de a uno, medir, documentar y considerar trade-offs.
- Block size, extents y freespace son parámetros de rendimiento, no solo administrativos.

BIBLIOGRAFÍA

BIBLIOGRAFÍA

Reyna Monteverde, T. (s. f.). Administración de Bases de Datos: Diseño interno, costo de acceso, consultas críticas y metodología de optimización. Universidad Nacional de Ingeniería.

PostgreSQL Global Development Group. (s. f.). Documentation: EXPLAIN; Using EXPLAIN.
<https://www.postgresql.org/docs/current/sql-explain.html> ; <https://www.postgresql.org/docs/current/using-explain.html>

Microsoft. (2026). Execution Plan Overview (SQL Server). <https://learn.microsoft.com/en-us/sql/relational-databases/performance/execution-plans>

Oracle/MySQL. (s. f.). MySQL Reference Manual: EXPLAIN Output Format.
<https://dev.mysql.com/doc/en/explain-output.html>

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
<https://www.db-book.com/>

Date, C.J. (2003). An Introduction to Database Systems (8.^a ed.). Addison-Wesley. ISBN: 978-0321197849

GRACIAS
TOTALES

